

CHƯƠNG 3

ĐIỀU PHỐI HỆ THỐNG VỚI DOCKER-COMPOSE

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Hiểu kiến trúc đa dịch vụ của các ứng dụng.
- Sử dụng Networking và Environment Variables hiệu quả.
- Thiết kế và triển khai hệ thống đa dịch vụ bằng Docker Compose.
- Hiểu một số khái niệm của Kubernetes và cơ bản của K8s (Pod, Deployment, Service).
- Triển khai điều phối hệ thống đơn giản trên môi trường local.

3.1 Vai trò của docker-compose trong triển khai đa dịch vụ

3.1.1 Giới thiệu về docker-compose

Docker Compose là công cụ giúp định nghĩa và chạy các ứng dụng Docker sử dụng nhiều container (multi-container) cùng lúc. Nó sử dụng một file cấu hình duy nhất để thiết lập các thành phần như database hay web server, từ đó ta có thể khởi động toàn bộ hệ thống chỉ bằng một câu lệnh.

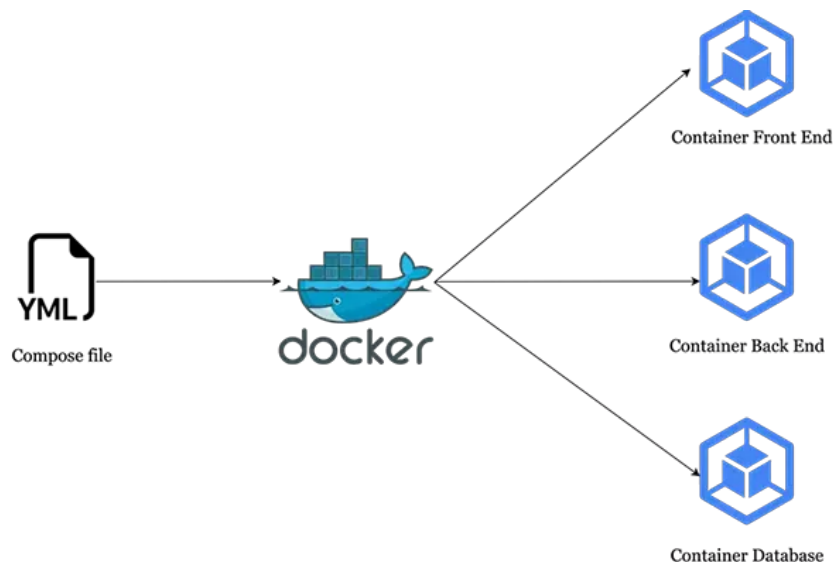
Thực tế khi xây dựng các ứng dụng dạng microservice sẽ có một vài dịch vụ cùng chạy, khi đó thường không chỉ chạy một container duy nhất mà cần kết hợp nhiều service khác nhau (ví dụ: một container cho ứng dụng Node.js, một container cho cơ sở dữ liệu MongoDB, và một container cho Redis). Việc gõ lệnh chạy từng container riêng lẻ và thiết lập mạng (network) hay ổ đĩa (volume) cho chúng rất phức tạp.

Chính vì vậy, Docker Compose ra đời để giải quyết vấn đề này. Nó đóng vai trò như một "nhạc trưởng" điều phối các container. Với compose quản trị viên sử dụng file YAML để thực hiện cấu hình các services cho ứng dụng. Sau đó dùng lệnh để tạo và thực thi những cấu hình đó, các bước cơ bản bao gồm:

- Khai báo môi trường hoạt động của ứng dụng trong Dockerfile.
- Khai báo các dịch vụ cần thiết để chạy ứng dụng trong file `docker-compose.yml`.
- Thực thi câu lệnh `docker-compose up` để khởi tạo và chạy toàn bộ các dịch vụ liên quan tới ứng dụng.

Hình trên mô tả cách thức làm việc của docker compose, ta có thể thấy từ tập tin cấu hình YAML, docker engine sẽ tạo và chạy các container tương ứng cho mỗi dịch vụ trong chương trình ứng dụng. Các container này được thiết lập để hoạt động và có thể trao đổi được dữ liệu với nhau bên trong một network khép kín, đảm bảo tính cô lập với môi trường bên ngoài, cũng như không chịu ảnh hưởng từ điều kiện của máy host. Điều này không chỉ tiết kiệm thời gian mà còn đảm bảo tính nhất quán giữa các môi trường.

Các thành phần cơ bản của Docker Compose:



Hình 3.1: Mô hình hoạt động của docker compose

- **Compose file:** Đây là một tệp YAML (`docker-compose.yml`) chứa thông tin cần thiết về container để triển khai ứng dụng.
- **Các dịch vụ (services):** Là một nhóm container mà ta muốn tạo ra. Mỗi service đại diện cho một thành phần của ứng dụng, ví dụ như service backend, service frontend, service database...
- **Container:** Là một đối tượng chứa ứng dụng và các thành phần của nó, bao gồm phần mềm và cấu hình của ứng dụng. Docker Compose sử dụng các container để triển khai các services được định nghĩa trong tệp `docker-compose.yml`.
- **Các mạng (networks):** Định nghĩa các mạng ảo để các container trong cùng một service hoặc các service khác nhau liên lạc với nhau. Các mạng này cho phép các container truy cập lẫn nhau bằng tên hoặc địa chỉ IP.
- **Các khối lưu trữ (volumes):** Là một phần quan trọng của quá trình triển khai Docker Compose, cho phép lưu trữ dữ liệu của ứng dụng của mình độc lập với các container. Điều này giúp cho việc sao lưu, phục hồi và quản lý dữ liệu trở nên dễ dàng hơn.
- **Các biến môi trường (environment variables):** Là các biến môi trường được sử dụng để cấu hình các container của dịch vụ. Điều này giúp ta dễ dàng định nghĩa các cấu hình của ứng dụng của mình trong tệp `docker-compose.yml` một cách linh hoạt hơn.
- **Command line interface (CLI):** Là giao diện dòng lệnh để chạy các lệnh liên quan đến Docker Compose, cho phép tạo, triển khai, quản lý và xóa các container và service.

Như vậy, Docker compose giúp tạo ra nhiều môi trường độc lập (isolated environments) trong một host: Compose giúp cô lập môi trường hoạt động của các project luôn đảm bảo chúng không bị xung đột với nhau, cũng như tạo những bản sao của một môi trường nào đó một cách nhanh chóng và đơn giản nhất.

Bên cạnh đó, chỉ tạo lại các container khi chúng đã thay đổi: Compose giúp nhận biết các service chưa thay đổi và sử dụng lại các container tương ứng với các service đó.

Hỗ trợ điều chỉnh các biến sử dụng cho các môi trường: Compose dùng các biến

trong Compose file cho các môi trường. Vì vậy với môi trường hay người dùng khác nhau, có thể dễ dàng điều chỉnh các biến khi dùng Compose để thiết lập các service.

3.1.2 Ưu điểm và hạn chế của docker compose

Sử dụng Docker Compose mang lại nhiều lợi ích thiết thực, giúp lập trình viên và kỹ sư DevOps tối ưu hóa quy trình làm việc. Tuy nhiên, như bất kỳ công cụ nào, nó cũng có những hạn chế cần xem xét để áp dụng hiệu quả.

Ưu điểm nổi bật của docker compose

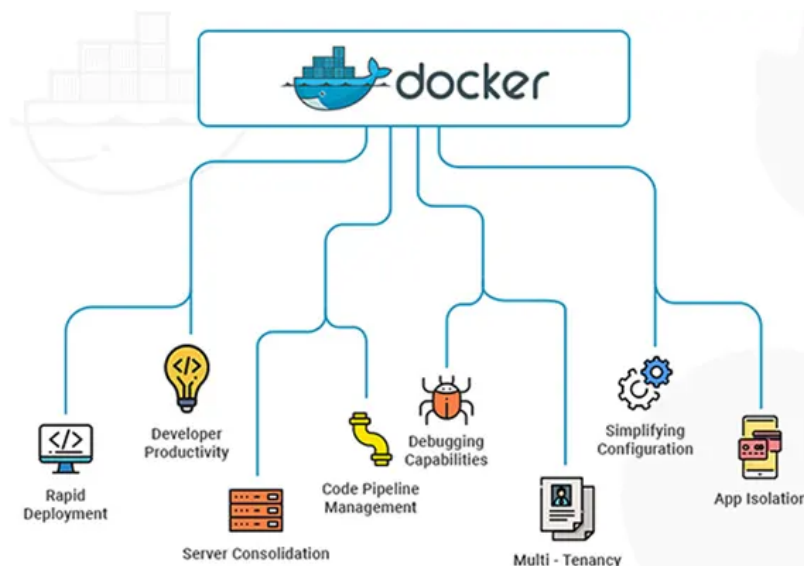
Một số ưu điểm khi sử dụng docker compose trong quản lý và triển khai ứng dụng đa dịch vụ đó là:

- **Đơn giản hóa quy trình:** Với Docker Compose, ta chỉ cần một file cấu hình duy nhất – `docker-compose.yml` – để định nghĩa và quản lý toàn bộ hệ thống container. Thay vì nhập hàng loạt lệnh phức tạp như khi dùng Docker riêng lẻ, một lệnh `docker-compose up` là đủ để khởi động tất cả dịch vụ. Điều này đặc biệt hữu ích khi làm việc với các ứng dụng đa thành phần, như web server, backend API và cơ sở dữ liệu.
- **Tính nhất quán giữa các môi trường:** Docker Compose đảm bảo rằng môi trường phát triển trên máy local giống hệt với môi trường kiểm thử hay sản xuất. Ta không còn phải lo lắng về sự khác biệt cấu hình hay phụ thuộc giữa các máy, từ đó giảm thiểu lỗi ”chạy được trên máy tôi nhưng không chạy trên server”.
- **Hỗ trợ phát triển nhóm:** File YAML của Docker Compose dễ dàng chia sẻ giữa các thành viên trong nhóm. Chỉ cần gửi file này, đồng nghiệp của ta có thể tái tạo toàn bộ hệ thống trên máy của họ mà không cần cài đặt thủ công từng phần. Điều này giúp tăng hiệu quả làm việc nhóm, đặc biệt trong các dự án lớn.
- **Tích hợp microservices hiệu quả:** Khi làm việc với một ứng dụng microservices, ta có thể dùng Docker Compose để chạy đồng thời nhiều dịch vụ mà không lo xung đột. Ví dụ, một hệ thống với frontend (React), backend (Node.js) và database (PostgreSQL) có thể được khởi động cùng lúc, với các kết nối mạng được thiết lập tự động thông qua cấu hình Compose.
- **Tiết kiệm thời gian phát triển:** Thay vì mất hàng giờ để thiết lập môi trường, Docker Compose cho phép ta tập trung vào việc viết mã và kiểm thử. Công cụ này tự động xử lý các vấn đề như ánh xạ cổng, gắn volume, và quản lý biến môi trường, giúp lập trình viên đẩy nhanh tiến độ dự án.

Hạn chế khi sử dụng

Bên cạnh những ưu điểm vượt trội, Docker Compose cũng có một số hạn chế mà người dùng cần cân nhắc:

- **Không phù hợp với hệ thống quy mô lớn:** Docker Compose được thiết kế chủ yếu cho phát triển và triển khai ở quy mô nhỏ đến trung bình. Khi ứng dụng của chúng ta mở rộng với hàng chục hoặc hàng trăm container, cùng yêu cầu về tính sẵn sàng cao (high availability) và tự động mở rộng (auto-scaling), thì Kubernetes hoặc Docker Swarm sẽ là lựa chọn phù hợp hơn.
- **Tiêu tốn tài nguyên:** Dù nhẹ hơn máy ảo, việc chạy nhiều container cùng lúc với Docker Compose vẫn đòi hỏi máy chủ có đủ RAM và CPU. Nếu không quản lý tốt, hệ thống có thể bị quá tải, đặc biệt trên các máy cấu hình thấp.



Hình 3.2: Ưu điểm và hạn chế khi sử dụng docker compose

- **Hạn chế về bảo mật:** File `docker-compose.yml` thường chứa các thông tin nhạy cảm như mật khẩu database hoặc API key. Nếu không được mã hóa hoặc quản lý cẩn thận, những thông tin này có thể bị lộ khi chia sẻ file, gây rủi ro cho dự án.
- **Thiếu tính năng quản lý nâng cao:** So với các công cụ như Kubernetes, Docker Compose không cung cấp các tính năng như tự động phục hồi (self-healing), cân bằng tải (load balancing), hay giám sát chi tiết. Điều này khiến nó kém linh hoạt trong các môi trường sản xuất phức tạp.

Do đó, để tận dụng tối đa Docker Compose, ta cần hiểu rõ bối cảnh áp dụng công cụ này lý tưởng cho:

- **Giai đoạn phát triển:** Tạo môi trường local nhanh chóng để thử nghiệm ứng dụng.
- **Dự án nhỏ và vừa:** Quản lý hệ thống với số lượng container hạn chế.
- **Kiểm thử:** Mô phỏng môi trường sản xuất để chạy test tự động.

Ngược lại, nếu dự án yêu cầu khả năng mở rộng lớn hoặc tính sẵn sàng cao, hãy cân nhắc chuyển sang các giải pháp thay thế như Kubernetes sau khi đã hoàn thiện giai đoạn phát triển với Docker Compose.

3.1.3 Cú pháp sử dụng của docker compose

Các lệnh thường được sử dụng để khởi động hoặc dừng các dịch vụ trong docker compose:

Ví dụ ta có file `compose.yml` với nội dung như sau:

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
```

Chương 3. Điều phối hệ thống với Docker-compose

Bảng 3.1: Các lệnh khởi động và dừng dịch vụ

Lệnh	Mô tả	Ví dụ
<code>docker compose up</code>	Khởi động tất cả service	<code>docker compose up</code>
<code>docker compose up -d</code>	Khởi động ở chế độ background (khuyến nghị)	<code>docker compose up -d</code>
<code>docker compose up --build</code>	Build lại image rồi khởi động	<code>docker compose up --build</code>
<code>docker compose down</code>	Dừng và xóa container	<code>docker compose down</code>
<code>docker compose down -v</code>	Dừng và xóa cả volume	<code>docker compose down -v</code>
<code>docker compose restart</code>	Khởi động lại tất cả service	<code>docker compose restart</code>

```
- REDIS_PORT=${REDIS_PORT}

redis:
  image: redis:alpine
```

Trong file cấu hình trên, chúng ta định nghĩa hai dịch vụ:

- Dịch vụ web sử dụng image được biên dịch từ Dockerfile trong thư mục hiện hành, ánh xạ địa chỉ cổng của ứng dụng sang port 5000. Bên cạnh đó còn định nghĩa một số biến môi trường được dùng trong quá trình chạy dịch vụ Redis.
- Dịch vụ Redis sử dụng một image được lấy từ Docker hub để build.

Để khởi tạo và chạy hai dịch vụ trên, ta thực hiện câu lệnh sau:

```
$docker compose up
```

Chỉ với một lệnh duy nhất, ta có thể tạo và khởi động tất cả các dịch vụ từ tệp cấu hình của mình. Compose sẽ xây dựng image cho dịch vụ web, tải image Redis và khởi động cả hai container.

Sau khi các dịch vụ được khởi động, ta có thể truy cập vào link <http://localhost:8000> để vào ứng dụng web.

Để dừng các dịch vụ và xóa các container liên quan, ta thực hiện lệnh sau:

```
$docker compose down
```

3.1.4 Các tùy chọn thường dùng với compose

Các lệnh thường được sử dụng để build và scale (mở rộng) phạm vi trong docker compose:

Lệnh dùng xem trạng thái và log của các container trong quá trình hoạt động:

Trong quá trình sử dụng, ta có thể tương tác trực tiếp với các container để truy cập vào bên trong và sử dụng shell:

Bảng 3.2: Các lệnh build và scale

Lệnh	Mô tả
<code>docker compose build</code>	Build tất cả image
<code>docker compose build web</code>	Build riêng service web
<code>docker compose build --no-cache</code>	Build không dùng cache
<code>docker compose up -d --scale web=3</code>	Scale service web lên 3 instance

Bảng 3.3: Các lệnh xem trạng thái và log

Lệnh	Mô tả
<code>docker compose ps</code>	Xem danh sách container đang chạy
<code>docker compose logs</code>	Xem log của tất cả service
<code>docker compose logs -f</code>	Xem log realtime (follow)
<code>docker compose logs web</code>	Xem log của service tên web
<code>docker compose logs -f --tail=100 api</code>	Xem 100 dòng log cuối của service api

Bảng 3.4: Các lệnh tương tác với container

Lệnh	Mô tả
<code>docker compose exec web sh</code>	Vào terminal của service web (khuyến nghị)
<code>docker compose exec web bash</code>	Vào bash nếu có
<code>docker compose run web sh</code>	Tạo container mới để chạy lệnh (không map port)
<code>docker compose top</code>	Xem tiến trình đang chạy trong các container

Bảng 3.5: Các lệnh tiện ích khác

Lệnh	Mô tả
<code>docker compose config</code>	Kiểm tra file <code>docker-compose.yml</code> có lỗi không
<code>docker compose config --services</code>	Liệt kê tất cả service
<code>docker compose stop</code>	Dừng container nhưng không xóa
<code>docker compose start</code>	Khởi động lại container đã stop
<code>docker compose rm</code>	Xóa container (sau khi down)
<code>docker compose pull</code>	Tải image mới nhất từ registry

Một số tập lệnh hữu ích thường dùng khác như:

Ví dụ sau minh họa cho việc sử dụng các lệnh để tạo và tương tác với container trong docker compose:

```
# Khởi động lại một service cụ thể
docker compose restart web

# Xem log + theo dõi realtime
docker compose logs -f web

# Vào container để debug
docker compose exec web sh

# Build lại và khởi động lại service
docker compose up -d --build web

# Dừng hết, xóa volume và cache, khởi động mới hoàn toàn
docker compose down -v --rmi all && docker compose up -d --build
```

3.1.5 So sánh với Kubernetes

Kubernetes là một hệ thống mã nguồn mở giúp tự động hóa việc triển khai, nhân rộng và quản lý các ứng dụng container. Đây là hệ thống có tính khả chuyển cao, có thể mở rộng để quản lý các ứng dụng được đóng gói và các service, giúp thuận lợi trong việc cấu hình và tự động hóa việc triển khai ứng dụng. Kubernetes là một hệ sinh thái lớn và phát triển nhanh chóng. Các dịch vụ, sự hỗ trợ và công cụ có sẵn rộng rãi.

Tên gọi Kubernetes có nguồn gốc từ tiếng Hy Lạp, có ý nghĩa là người lái tàu hoặc hoa tiêu. Google mở mã nguồn Kubernetes từ năm 2014. Kubernetes xây dựng dựa trên một thập kỷ rưỡi kinh nghiệm mà Google có được với việc vận hành một khối lượng lớn workload trong thực tế, kết hợp với các ý tưởng và thực tiễn tốt nhất từ cộng đồng.

Các container là một cách tốt để đóng gói và chạy các ứng dụng của bạn. Trong môi trường production, bạn cần quản lý các container chạy các ứng dụng và đảm bảo rằng không có khoảng thời gian downtime. Ví dụ, nếu một container bị tắt đi, một container khác cần phải khởi động lên. Điều này sẽ dễ dàng hơn nếu được xử lý bởi một hệ thống.



Hình 3.3: Kubernetes (K8s) nền tảng mã nguồn mở

Kubernetes cung cấp cho người dùng một framework để chạy các hệ phân tán một cách mạnh mẽ. Nó đảm nhiệm việc nhân rộng và chuyển đổi dự phòng cho ứng dụng, cung cấp các mẫu deployment và hơn thế nữa. Ví dụ, Kubernetes có thể dễ dàng quản lý một triển khai công nghệ cho hệ thống.

Kubernetes cung cấp cho chúng ta những thứ sau:

- **Service discovery và cân bằng tải:** Kubernetes có thể expose một container sử dụng DNS hoặc địa chỉ IP của riêng nó. Nếu lượng traffic truy cập đến một container cao, Kubernetes có thể cân bằng tải và phân phối lưu lượng mạng (network traffic) để việc triển khai được ổn định.
- **Điều phối bộ nhớ:** Kubernetes cho phép người dùng tự động mount một hệ thống lưu trữ mà họ chọn, như local storages, public cloud providers, v.v.
- **Tự động rollouts và rollbacks:** Người dùng có thể mô tả trạng thái mong muốn cho các container được triển khai dùng Kubernetes và nó có thể thay đổi trạng thái thực tế sang trạng thái mong muốn với tần suất được kiểm soát. Ví dụ, lập trình viên có thể tự động hoá Kubernetes để tạo mới các container cho việc triển khai của họ, xoá các container hiện có và áp dụng tất cả các resource của chúng vào container mới.
- **Đóng gói tự động:** Chúng ta cung cấp cho Kubernetes một cluster gồm các node mà nó có thể sử dụng để chạy các tác vụ được đóng gói (containerized task). Đồng thời, cho Kubernetes biết mỗi container cần bao nhiêu CPU và bộ nhớ (RAM). Kubernetes có thể điều phối các container đến các node để tận dụng tốt nhất các resource của hệ thống.
- **Tự phục hồi:** Kubernetes khởi động lại các containers bị lỗi, thay thế các container, xoá các container không phản hồi lại cấu hình health check do người dùng xác định và không cho các client biết đến chúng cho đến khi chúng sẵn sàng hoạt động.
- **Quản lý cấu hình và bảo mật:** Kubernetes cho phép người dùng lưu trữ và quản lý các thông tin nhạy cảm như: password, OAuth token và SSH key. Bạn có thể triển khai và cập nhật lại secret và cấu hình ứng dụng mà không cần build lại các container image và không để lộ secret trong cấu hình stack của chính họ.

Dưới đây là bảng so sánh chi tiết và khách quan giữa Docker Compose và Kubernetes theo một số tiêu chí:

Bảng 3.6: So sánh Docker Compose và Kubernetes

Tiêu chí	Docker Compose	Kubernetes (K8s)
Mục đích chính	Định nghĩa và chạy đa container trên một máy	Orchestration container trên nhiều máy (cluster)
Phạm vi sử dụng	Development, Testing, Demo, Production nhỏ	Production lớn, hệ thống phân tán cao
Độ phức tạp	Rất đơn giản, dễ học	Rất phức tạp, dốc học cao
Cách cài đặt	Cài Docker là dùng được	Cần cluster (Minikube, Kind, EKS, GKE, AKS...)
File cấu hình	<code>docker-compose.yml</code> (YAML đơn giản)	Nhiều file YAML (Deployment, Service, Ingress, ConfigMap, Secret...)
Scaling	Scale thủ công (<code>--scale</code>) hoặc đơn giản	Tự động scaling (Horizontal Pod Autoscaler)
High Availability	Không hỗ trợ tốt (chỉ 1 máy)	Hỗ trợ mạnh (self-healing, replica, failover)
Load Balancing	Cơ bản (built-in)	Mạnh mẽ với Service + Ingress + LoadBalancer

Chương 3. Điều phối hệ thống với Docker-compose

Tiêu chí	Docker Compose	Kubernetes (K8s)
Networking	Bridge network đơn giản	Mạng overlay phức tạp, linh hoạt cao
Storage	Volume cơ bản	PersistentVolume, StorageClass, Dynamic Provisioning
Rolling Update / Roll-back	Hạn chế	Hỗ trợ mạnh, zero-downtime
Secret & Config	Dùng <code>.env</code> và <code>environment</code>	ConfigMap + Secret (an toàn và chuyên nghiệp)
Monitoring	Cơ bản (logs)	Tích hợp Prometheus, Grafana, ELK...
Chi phí	Thấp (chạy trên 1 server)	Cao hơn (cần nhiều server hoặc cloud)
Learning Curve	Dễ, phù hợp beginner	Khó, cần thời gian học sâu
Quản lý	Manual hoặc script đơn giản	Tự động hóa cao, khai báo (Declarative)

Dựa trên sự so sánh, ta có thể đưa ra một số khuyến nghị về điều kiện khi nào sử dụng giải pháp nào như sau:

Bảng 3.7: Khuyến nghị lựa chọn theo tình huống

Tình huống	Khuyến nghị
Phát triển local, test, demo	Docker Compose
Dự án nhỏ, dưới 10 services	Docker Compose
Production nhỏ (1-2 server)	Docker Compose hoặc Docker Swarm
Production trung bình & lớn	Kubernetes
Cần scale tự động, high availability	Kubernetes
Team có DevOps mạnh	Kubernetes
Startup giai đoạn đầu	Docker Compose
Doanh nghiệp lớn, hệ thống phân tán	Kubernetes

3.2 Điều phối hệ thống với docker-compose

Nội dung phần này sẽ hướng dẫn triển khai ứng dụng CMS mã nguồn mở Wordpress đa dịch vụ với docker compose.

Thành phần của hệ thống bao gồm các dịch vụ sau đây:

- WordPress (web application – frontend + PHP).
- MySQL (cơ sở dữ liệu).
- Redis (cache – minh họa service cache, WordPress có thể sử dụng làm object cache qua plugin nếu cần mở rộng sau).

Trong đó, tất cả các dịch vụ chạy trong một network độc lập (`wp-network`). Chúng ta sẽ sử dụng Dockerfile để build custom image cho WordPress.

Cấu trúc thư mục dự án được bố trí như sau:

```
wordpress-docker-lab/  
├── docker-compose.yml
```

Chương 3. Điều phối hệ thống với Docker-compose

— .env	
— Dockerfile	# Dockerfile cho WordPress (custom image)
— init.sql	# (tùy chọn) script khởi tạo DB
— README.md	# (bạn có thể tạo để ghi chú)

File 1: .env (lưu trữ Environment Variables – bảo mật, không commit lên Git), nội dung file .env như sau:

```
# MySQL
MYSQL_ROOT_PASSWORD=SuperSecretRootPass123
MYSQL_DATABASE=wordpress
MYSQL_USER=wp_user
MYSQL_PASSWORD=wp_password_strong_456

# WordPress
WORDPRESS_DB_HOST=db
WORDPRESS_DB_USER=wp_user
WORDPRESS_DB_PASSWORD=wp_password_strong_456
WORDPRESS_DB_NAME=wordpress

# Redis (cache)
REDIS_PASSWORD=redis_secure_pass_789
```

Có thể thấy trong file .env chứa rất nhiều thông tin nhạy cảm và quan trọng, cần được bảo mật tốt. Do vậy với các thông tin này, thường sẽ sử dụng các file .env cho từng môi trường khác nhau (dev, stage, production) để tăng tính bảo mật cho hệ thống.

File 2: Dockerfile (dùng để build custom image cho WordPress)

Đây là file chứa nội dung để build image, ở đây ta sử dụng luôn image được WordPress cung cấp sẵn trên Docker Hub, chứa toàn bộ mã nguồn của CMS này bên trong:

```
# Sử dụng image chính thức của WordPress làm base
FROM wordpress:6.6.1-php8.3-apache

# Cài đặt Memcached extension (tùy chọn)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libmemcached-dev \
    zlib1g-dev \
    libssl-dev \
    && pecl install memcached \
    && docker-php-ext-enable memcached \
    && apt-get purge -y --auto-remove zlib1g-dev libssl-dev \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/* /tmp/pear/*

# Đặt quyền thư mục
RUN chown -R www-data:www-data /var/www/html

EXPOSE 80
```

Ở đây ta thấy có lệnh copy file cấu hình wp-config.php, do đó cần đảm bảo file đó phải tồn tại ở thư mục gốc để tránh bị lỗi khi build image.

Tiếp theo sẽ tạo file cho docker compose:

File 3: docker-compose.yml (file chính định nghĩa toàn bộ hệ thống)

```
version: '3.9'

services:
  db:
    image: mysql:8.0
    container_name: wp-mysql
    restart: unless-stopped
    env_file:
      - .env
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql  # script khởi tạo DB (nếu có)
    networks:
      - wp-network
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root
↪ -p${MYSQL_ROOT_PASSWORD} --silent"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
  redis:
    image: redis:7.2-alpine
    container_name: wp-redis
    restart: unless-stopped
    command: redis-server --requirepass ${REDIS_PASSWORD}
    env_file:
      - .env
    volumes:
      - redis_data:/data
    networks:
      - wp-network

  wordpress:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: wp-app
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    condition: service_healthy
    redis:
      condition: service_started
    ports:
      - "8080:80"
    env_file:
      - .env
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: ${MYSQL_USER}
      WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
      WORDPRESS_DB_NAME: ${MYSQL_DATABASE}
```

Chương 3. Điều phối hệ thống với Docker-compose

```
# Minh họa env cho Redis (WordPress có thể dùng sau khi cài plugin Redis Object
↪ Cache)
WORDPRESS_REDIS_HOST: redis
WORDPRESS_REDIS_PASSWORD: ${REDIS_PASSWORD}
volumes:
  - wp_data:/var/www/html
networks:
  - wp-network

# Định nghĩa network độc lập
networks:
  wp-network:
    driver: bridge
    name: wp-network
    #internal: false # false = có thể truy cập từ host

# Định nghĩa volume để lưu trữ dữ liệu bền vững
volumes:
  db_data:
  redis_data:
  wp_data:
```

Có thể thấy trong file cấu hình thông tin tạo network riêng cho hệ thống và khai báo các volume riêng để lưu trữ dữ liệu độc lập cho từng dịch vụ. Các biến môi trường được định nghĩa để truy cập thông tin từ `.env`, sau đó truyền vào các dịch vụ.

Lưu ý: Từ khóa `depends_on` được sử dụng nhằm đảm bảo thứ tự khởi động của các dịch vụ, `restart: unless-stopped` giúp dịch vụ tự động phục hồi khi gặp sự cố phải khởi động lại.

Sau khi mọi thứ sẵn sàng, tại thư mục gốc của dự án, hãy thực hiện lệnh sau:

```
$docker compose up -d --build
```

Kiểm tra các container đã được khởi tạo đầy đủ chưa bằng lệnh:

```
$docker compose ps
```

Ta có thể kiểm tra log của container trong trường hợp bị lỗi:

```
$docker compose logs -f wordpress
```

Sau khi các container đã được khởi tạo và chạy, để truy cập đến ứng dụng WordPress, hãy mở trình duyệt và gõ địa chỉ: **`http://localhost:8080`**. Tiếp theo cần hoàn tất cài đặt WordPress (chọn ngôn ngữ, tạo tài khoản admin).

Khi thực hiện lệnh `docker compose down`, các container sẽ được loại bỏ khỏi hệ thống:

Một số lệnh hữu ích để quản lý các container:

```
=> => unpacking to docker.io/library/wp-docker-wordpress:latest
=> resolving provenance for metadata file
[+] up 6/6
♦ Image wp-docker-wordpress Built
♦ Network wp-network Created
♦ Container wp-redis Started
♦ Container wp-mysql Started
♦ Container wp-phpmyadmin Started
♦ Container wp-app Started
```

Hình 3.4: Các containers được start, network được khởi tạo

```
anhchuyen@linuxserver:~/wp-docker$ docker compose ps
WARN[0000] /home/anhchuyen/wp-docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
wp-app              wp-docker-wordpress "docker-entrypoint.s..." wordpress  7 seconds ago   Up 6 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
wp-mysql            mysql:8.0            "docker-entrypoint.s..." db         7 seconds ago   Up 6 seconds   3306/tcp, 33060/tcp
wp-phpmyadmin       phpmyadmin:latest    "docker-entrypoint.s..." phpmyadmin 7 seconds ago   Up 6 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
wp-redis            redis:7.2-alpine     "docker-entrypoint.s..." redis      7 seconds ago   Up 6 seconds   6379/tcp
```

Hình 3.5: Danh sách các container đang hoạt động

```
anhchuyen@linuxserver:~/wp-docker$ docker compose down
WARN[0000] /home/anhchuyen/wp-docker/docker-compose.yml: the attr
[+] down 5/5
♦ Container wp-phpmyadmin Removed
♦ Container wp-app Removed
♦ Container wp-mysql Removed
♦ Container wp-redis Removed
♦ Network wp-network Removed
anhchuyen@linuxserver:~/wp-docker$ docker compose ps
```

Hình 3.6: Thực hiện lệnh xóa bỏ các container, network

Bảng 3.8: Các lệnh quản lý container hữu ích

Lệnh	Mô tả
<code>docker compose down</code>	Dừng và xóa container
<code>docker compose down -v</code>	Dừng và xóa cả volume
<code>docker compose up -d --build</code>	Build lại image và chạy
<code>docker compose logs wordpress</code>	Xem log của service WordPress
<code>docker compose exec wordpress</code>	Vào shell bên trong container WordPress
<code>bash</code>	

Bài thực hành

Bài 1

Hãy viết file YAML để cấu hình tự động với docker compose cho ứng dụng web trong bài 2.

Bài 2

Thay đổi các thông số, mật khẩu trong `.env` của bài thực hành trên, sau đó build lại image và khởi tạo compose từ đầu, kiểm tra và xác nhận hệ thống hoạt động.

Bài 3

Thay đổi cổng dịch vụ web để Wordpress có thể chạy trực tiếp trên cổng 80.

Bài 4 (nâng cao)

Thêm phpMyAdmin service để quản lý DB. Cài plugin Redis Object Cache cho WordPress để sử dụng Redis thực sự.

Bài 5

Thay MySQL bằng PostgreSQL (chỉ cần đổi image và env variables). Triển khai lên server (production) với Docker Compose.